

Ho Chi Minh City University of Technology



LINEAR ALGEBRA (MT1007)

Assignment Report

Randomized PCA and Large-Scale Data Dimensionality Reduction

Lecturer: Dang Thi Kim Nhung

Class: CC05

Group: Group 11

Group Members:

Nguyen Minh Long	2550671
Nguyen Do Minh Phuc	2550687
Tran Thien Phuc	2550690
Dinh Kiet	2550668
Le Xuan Phuc	2550686
Tran Nhut Nguyen	2550681
Nguyen Minh Hoang	2550059
Thai Binh Kim Phuoc	2551393

Ho Chi Minh City, May 2026

Contents

1	INTRODUCTION	3
1.1	Background and Motivation	3
1.2	Randomized PCA: A Paradigm Shift	3
1.3	Explanation of Randomized PCA	3
1.4	Objectives and Scope of the Report	4
1.5	Evaluation Metrics	4
2	Theoretical background	5
2.1	Dimensionality reduction	5
2.1.1	Dimensionality of data	5
2.1.2	Problems of high-dimensional data	5
2.1.3	Motivation	5
2.1.4	Types of dimensionality reduction	6
2.2	Principal component analysis (PCA)	6
2.2.1	Definition	6
2.2.2	Data centering	6
2.2.3	Covariance matrix	6
2.2.4	Eigendecomposition	7
2.2.5	SVD Formulation	7
2.3	Low-rank matrix approximation	7
2.3.1	Rank- k approximation	7
2.3.2	Eckart-Young theorem	8
2.3.3	Reconstruction error	8
2.4	Limitaion of classical PCA	8
2.4.1	Computational complexity	8
2.4.2	Memory cost	8
2.4.3	Scalability limitaiton	9
2.5	Randomized PCA	9
2.5.1	Definition	9
2.5.2	Approximate subspace construction	9
2.5.3	Advantages over classical PCA	10
3	Energy threshold for selecting k	10
4	ALGORITHM OR MATHEMATICAL MODEL EMPLOYED	10
4.1	Problem statement and theoretical basis	10
4.2	Algorithmic steps in mathematical form	11
4.2.1	Stage A: Finding an approximate subspace (range finder)	11
4.2.2	Stage B: Low-dimensional SVD	11
4.3	Model Control Parameters	12
4.4	Complexity Analysis	12
4.5	Dynamic dimension selection via energy threshold	12
4.6	Error evaluation via Frobenius norm	13
4.7	Randomized PCA Pseudocode	14

5	Results, Deliverables, and Experimental Evaluation	16
5.1	Main Libraries and Tools	16
5.2	Description of Key Functions and Modules	16
5.2.1	Data Ingestion and Preprocessing	17
5.2.2	Dynamic Energy Thresholding	17
5.2.3	Standard PCA Baseline	19
5.2.4	Randomized PCA Engine	19
5.2.5	Error Evaluation and Reconstruction	21
5.2.6	Visual Comparison	24
6	CONCLUSION	31

1 INTRODUCTION

1.1 Background and Motivation

In the current era of Big Data, the exponential growth of information in the world we inhabit has created unprecedented challenges for traditional analytical methods. Modern datasets are frequently characterized by extremely high dimensionality ($d \gg 1$) and massive sample sizes ($n \gg 1$), emerging in diverse fields ranging from genomics to natural language processing and computer vision.

The rapid transformation in the flow of history has compelled scientists to accelerate the development of new methodologies to meet the world's demands. This necessity has led to the birth and evolution of modern, agile, and characteristic data statistical systems. Among the many tools supporting us today, Principal Component Analysis (PCA) has long been regarded as a new standard for dimensionality reduction and feature extraction.

However, classical PCA algorithms based on full Singular Value Decomposition (SVD) carry a very high computational complexity, approximately $O(\min(n^2d, nd^2))$. When processing large-scale matrices, these methods become mathematically and computationally prohibitive due to memory constraints and execution time. This reality necessitates more efficient alternative solutions for both the present and the future.

1.2 Randomized PCA: A Paradigm Shift

To address the limitations of classical approaches, randomized algorithms were created and have gradually become a breakthrough solution across various fields. Instead of performing exact computations on the entire data space, Randomized PCA utilizes random projections to identify a low-dimensional subspace while still retaining the most critical information from the original matrix.

The theoretical foundation of this method lies in low-rank approximation. By combining probability theory with available linear algebra, Randomized PCA significantly reduces computational costs to approximately $O(nd \log(k))$, where k is the target number of principal components. This method is not only faster but also possesses excellent scalability for distributed computing environments, helping to alleviate the pressure on all parties involved in complex computations.

1.3 Explanation of Randomized PCA

The core idea of Randomized PCA is to use a randomized approach to compute a near-optimal low-rank approximation of a large matrix A . This process is divided into main phases:

- **Step 1: Random Projection.** Generate a random Gaussian matrix Ω and compute $Y = A\Omega$.
- **Step 2: Orthonormal Basis.** Compute the QR decomposition of Y to obtain Q .
- **Step 3: Dimensionality Reduction.** Project the original matrix A onto the smaller subspace: $B = Q^T A$.
- **Step 4: Standard SVD.** Compute SVD on the small matrix B : $B = \tilde{U}\Sigma V^T$.
- **Step 5: Final Reconstruction.** Recover the principal components: $U = Q\tilde{U}$.

1.4 Objectives and Scope of the Report

This report focuses on investigating and implementing Randomized PCA for large-scale data dimensionality reduction. According to the project requirements, our main objectives include:

- Presenting the theoretical foundation of PCA and SVD.
- Analyzing the limitations of classical PCA for large-scale data.
- Constructing the Randomized PCA algorithm for large matrices.
- Implementing an algorithm to select the optimal number of dimensions k based on an energy threshold η :

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq \eta$$

- Evaluating approximation error using the Frobenius norm $\|A - A_k\|_F$ and comparing execution times.

1.5 Evaluation Metrics

To assess the performance and accuracy of the Randomized PCA algorithm, we employ two primary quantitative metrics as defined in the project requirements.

1. Energy Threshold for Dimension Selection The number of retained dimensions k is not chosen arbitrarily but is determined by the cumulative energy threshold η . This ensures that the selected principal components capture a sufficient proportion of the total variance in the dataset:

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq \eta$$

where σ_i represents the i -th singular value, and r is the full rank of the matrix. Typically, η is set between 0.9 and 0.99 depending on the required precision.

2. Approximation Error via Frobenius Norm The fidelity of the dimensionality reduction is measured using the Frobenius norm of the difference between the original matrix A and its low-rank approximation A_k :

$$\text{Error} = \|A - A_k\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij} - (a_k)_{ij}|^2}$$

A smaller Frobenius norm indicates that the randomized subspace effectively preserves the structural integrity of the large-scale data.

3. Computational Efficiency In addition to accuracy, we evaluate the execution time (T_{exec}) of Randomized PCA compared to the classical SVD-based PCA. The primary goal is to demonstrate a significant reduction in processing time while maintaining an acceptable error margin for datasets exceeding 100,000 samples.

2 Theoretical background

2.1 Dimensionality reduction

2.1.1 Dimensionality of data

When considering the health of a single human, it is necessary to consider it in many aspects: height, weight, BMI, blood pressure, etc. Each of the aspects is called dimension of data. In other words, dimension of data is the number of features that is used to analyze an object. A set of data, represented by a matrix, denoted A , can contain as many dimensions as possible. It can be written as:

$$A \in R^{m \times n}$$

m : number of samples

n : number of features (dimensions)

2.1.2 Problems of high-dimensional data

In the reality, an object can be represented in thousand or million of dimensions. However, this large number of dimensions leads to many problems.

1) Cost-efficiency

The higher in dimension leads to the increment of calculation. Computers will need more RAM storage in order to execute that number of calculation.

2) Curse of dimensionality

This term is stated by Richard E. Bellman [1]. In such a high-dimensional space, the data points become sparse, making tasks such as data analysis and machine learning become more complicated. In addition, the distance between the points become approximately equal, which negatively affects the algorithms using this property [2], like K-Nearest Neighbor (KNN) or clustering.

3) Redundant features and noise

Some features in a data set have relationship with others. For instance, in daily life, the more amount of electricity the households use, the more money they need to pay. This is a basic proportional relationship. However, when we put these two assumptions in a same data set, it makes the data set become unnecessarily larger, as it is possible to just list one of them and conclude the other one. In addition, some features in a data set is not crucial enough to contribute to the whole analyzation. This is when we need to reduce the dimension of the data

2.1.3 Motivation

The aim of dimensionality reduction is to convert the original data set into a low-dimensional set while keeping the most important information. Mathematically, it is represented as:

$$n \rightarrow k, \quad k \ll n$$

where k is the reduced dimension. Dimensional reduction provides several advantages, including lower the computational price, reduced memory usage, improved visualization and reduced overfitting [2].

2.1.4 Types of dimensionality reduction

There are two types of dimensionality reduction:

- Feature selection: selecting a subset of original set
- Feature extraction: creating new features from original data

PCA is an example of feature extraction and it is one of the most common dimensionality reduction techniques.

2.2 Principal component analysis (PCA)

2.2.1 Definition

PCA is a statistic method used to reduce the dimensionality of the data while keeping as much variances as possible [3]. It works by finding orthogonal directions, called principle components, where the variance of the data set is maximized.

2.2.2 Data centering

Data centering is a preprocessing step in PCA. The matrix of data set A is centered by subtracting each value by the mean of their corresponding feature.

$$x'_i = x_i - \mu_j$$

where x_i is the original value on row i of the matrix, x'_i is the value of x_i after centering process, and μ_j is the mean of the feature of x_i corresponding to column j . This makes the mean of the values in each feature equal to 0. Data centering is a crucial part in preparing for the PCA algorithm. Without data centering, the finding of principle components becomes inaccurate.

For the remaining part of the section, A is referred as the centered version.

2.2.3 Covariance matrix

The covariance matrix of the centered data is defined as:

$$C = \frac{1}{m-1} A^T A$$
$$C = \begin{pmatrix} \text{cov}(x_1, x_1) & \text{cov}(x_1, x_2) & \dots & \text{cov}(x_1, x_n) \\ \text{cov}(x_2, x_1) & \text{cov}(x_2, x_2) & \dots & \text{cov}(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(x_n, x_1) & \text{cov}(x_n, x_2) & \dots & \text{cov}(x_n, x_n) \end{pmatrix}$$

Covariance matrix is symmetric ($C = C^T$) and positive semi-definite. This matrix contains the covariance of all pairs of features. As there are n features in matrix A , there are $n \times n$ pairs of features, so the covariance matrix is a $n \times n$ matrix. The main diagonal entries are the variances of each feature, therefore, the trace of the matrix is the total variance in the data.

$$\text{tr}(C) = \sum_{i=1}^n \sigma_i^2$$

where σ_i^2 is the variance of feature i .

2.2.4 Eigendecomposition

The eigenvectors of the covariance matrix show the direction of the principle components, while the corresponding eigenvalues represent the amount of variance on each direction [3]. Since the matrix C is symmetric and positive semi-definite, it can be written by using spectrum theorem:

$$C = V\Lambda V^T$$

where $V = [v_1 | v_2 | \dots | v_n]$ is an orthogonal matrix with each column v_i is an eigenvector, $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$ is a matrix contains eigenvalues organized in the descending order. The low-dimensional form of the data is obtained by the projection:

$$Z = AV_k$$

where $V_k = [v_1 | v_2 | \dots | v_k]$ contains k eigenvectors corresponding to the k largest eigenvalues.

2.2.5 SVD Formulation

Instead of calculating the covariance matrix, which can be a large calculation, in PCA, SVD (Singular Value Decomposition) is commonly used. SVD theorem: For any matrix $A \in R^{m \times n}$ with rank $r \leq \min(m, n)$, there is a decomposition:

$$A = U\Sigma V^T$$

where:

- $U \in R^{m \times m}$ is an orthogonal matrix containing columns called left singular vectors
- $V \in R^{n \times n}$ is an orthogonal matrix containing columns called right singular vectors
- $\Sigma \in R^{m \times n}$ is a diagonal matrix containing singular values σ_i organized in descending order, and the remaining entries are 0.

Connecting to eigendecomposition:

$$A^T A = V\Sigma^T U^T U \Sigma V^T = V (\Sigma^T \Sigma) V^T$$

Comparing with $C = V\Lambda V^T$, it is seen that $\lambda_i = \frac{\sigma_i^2}{m-1}$, and the right singular vectors of A are exactly the principle components. It is more common to use SVD instead of eigendecomposition of C because it avoids rounding error in computing $A^T A$ and is numerically more stable [4]. In practice, a thin SVD is preferred:

$$A_r = U_r \Sigma_r V_r^T$$

where $U_r \in R^{m \times r}$, $\Sigma_r \in R^{r \times r}$, $V_r \in R^{n \times r}$. These matrix are based on the size of Σ_r , which is Σ but eliminates the $m - r$ last rows.

2.3 Low-rank matrix approximation

2.3.1 Rank- k approximation

The main purpose of PCA is to represent a matrix of data in a low-dimension k matrix that contains most of the important information, where $k \ll n$. Given a data set represented by

matrix $A \in R^{m \times n}$ with full SVD $A = U\Sigma V^T$, the reduced rank- k approximation of the matrix A is defined as:

$$A_k = U_k \Sigma_k V_k^T$$

where only the first k largest singular values and the corresponding singular vectors are retained.

2.3.2 Eckart-Young theorem

Eckart-Young theorem is a justification for the low rank approximation [5]. Given any matrix $B \in R^{m \times n}$, $\text{rank}(B) \leq k$, the theorem states that:

$$\|A - A_k\|_F \leq \|A - B\|_F$$

In general, Eckart-Young theorem said that A_k is the best low rank approximation of A . There is no other rank- k approximation, regardless of their construction, can approximate A more accurate than A_k . It confirms that PCA, which based on SVD, is the optimal solution for dimensional reduction.

2.3.3 Reconstruction error

The reconstruction error is calculated by the Frobenius norm:

$$\|A - A_k\|_F^2 = \sum_{i=k+1}^r \sigma_i^2$$

The right side formula is the sum of the squared singular values that are eliminated. It is shown that when k increases, the sum on the right side decreases until reaching to 0, when $k = r$. The smaller the sum is, the more accurate the approximation is.

2.4 Limitaion of classical PCA

Although classical PCA has been proved to be the best way for dimensional reduction, it shows its disadvantages when dealing with large-scale data set.

2.4.1 Computational complexity

A full running of SVD in classical PCA for a matrix $A \in R^{m \times n}$ has a computational complexity of $O(mn^2)$ [6], which means that there are mn^2 calculation need to be executed. In a large-scale data set where there are hundreds or thousands or millions of values, this is a giant amount of calculation, which takes a lot of time to done all of that.

2.4.2 Memory cost

Running PCA requires the full data to be loaded into the memory. However, the problem is not about the data, but about the matrices that has been used to perform classical PCA. According to SVD formula, U has $m \times m$ elements, Σ has $m \times n$ elements and V has $n \times n$ elements. In a large-scale data set, m and n can be very large, making storing the three matrices to perform SVD becomes a big challenge. For example, if $m = 10^5$ and $n = 10^4$, the total storage for

SVD formula can be up to 80GB of RAM, clearly not appropriate in modern days where the price of RAM storage is rapidly increasing.

2.4.3 Scalability limitaiton

Beside complexity and cost problem, application of classical PCA is also an issue:

- Inability to perfectly execute real-time updated data: When new data is loaded, the algorithm will need to run again from scartch.
- When working with data sets that are naturally sparse, SVD cannot take advantage of this structure.
- Limited parallelism: SVD involves a long sequential of data dependencies, which makes it difficult to decompose the data for parallel calculation. These inconveniences leads to the development of randomized approximation method for SVD, of which randomized PCA is an example.

2.5 Randomized PCA

2.5.1 Definition

Randomized PCA is a matrix approximation technique for large-scale data set. The key idea is based on an observation that even though the data set is presented in n -dimensional space, the most important information actually lives in a much lower k -dimensional subspace, and the task is to approximate that subspace so that SVD will be performed on a much smaller matrix. Randomized PCA helps archiving this task.

2.5.2 Approximate subspace construction

Randomized PCA algorithm goes through 2 stages [6]. Stage 1: Randomized range finding

Given a random matrix $\Omega \in R^{n \times (k+p)}$, with p is the oversampling parameter (typically $p = 5$ or 10). The smaller sample matrix is constructed as:

$$Y = A\Omega$$

Each column of Y is a linear combination of the columns of A , so the range of Y is approximately equal to that of A_k

Apply QR decomposition to get an orthogonal basis:

$$Y = QR$$

where Q is an orthogonal matrix, R is an upper triangular matrix. Stage 2: SVD on reduced matrix

The original matrix is projected onto a lower dimensional subspace spanned by Q :

$$B = Q^T A \in R^{(k+p) \times n}$$

Compute SVD on smaller matrix B :

$$B = \tilde{U}\Sigma V^T$$

The left singular vectors are then reconstructed as:

$$U = Q\tilde{U}$$

The result is the approximate decomposition $A \approx U_k \Sigma_k V_k^T$ using k leading components.

2.5.3 Advantages over classical PCA

Randomized PCA offers a big improvement in many faces over classical PCA:

Criterion	Classical PCA	Randomized PCA
Complexity	$O(mn^2)$	$O(mn(k+p))$ with $k+p \ll \min(m, n)$
Memory	Full SVD of data	SVD of a smaller subspace
Large-scale data support	No	Yes
Parallelism	Limited	Efficient (BLAS matrix operations)
Approximation quality	Exact	Near-optimal

Note: BLAS is the abbreviation of Basic Linear Algebra Subprograms.

3 Energy threshold for selecting k

Choosing the right reduced dimension k is the most important decision in applying PCA, and one of the widely used criterion to choose k is the explained variance ratio. Explained variance is the amount of variance that a principle component can represent. The condition to select k is given by the ratio of the retained variance to the total variance:

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq \eta$$

where η is a desired amount of variance. Commonly, the threshold η is around 95%, meaning that 95% of the total information is kept by the principle components. This value is a balance between the compression of dimensions and the amount of retaining information. However, in real life, η can be changed based on the requirements of the job. It can increase if accuracy is necessary, or decrease to optimize the RAM storage.

4 ALGORITHM OR MATHEMATICAL MODEL EMPLOYED

4.1 Problem statement and theoretical basis

Given a large-scale data matrix $A \in \mathbb{R}^{m \times n}$, traditional PCA aims to find the Singular Value Decomposition (SVD):

$$A = U\Sigma V^T$$

Where:

- $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal matrices containing left and right singular vectors.
- $\Sigma \in \mathbb{R}^{m \times n}$ is a diagonal matrix containing singular values in descending order.

When m and n are extremely large, computing the full SVD is computational ($\mathcal{O}(\min(m^2n, mn^2))$). Randomized PCA solves this by finding a low-rank approximation matrix $Q \in \mathbb{R}^{m \times k}$ (where $k \ll \min(m, n)$) with orthonormal columns such that:

$$A \approx QQ^T A$$

This project implements a 2-stage randomized algorithm to achieve this approximation efficiently.

4.2 Algorithmic steps in mathematical form

The execution of Randomized PCA is divided into two main phases:

- Stage A (randomized range finder)
- Stage B (direct SVD on the reduced matrix)

4.2.1 Stage A: Finding an approximate subspace (range finder)

Generate a random Gaussian matrix: Create a random matrix $\Omega \in \mathbb{R}^{n \times 2k}$ (Where we slightly oversample to $2k$ or $k + s$ to ensure capturing the dominant subspace).

$$\Omega_{ij} \sim \mathcal{N}(0, 1)$$

Sample the column space: Form a sample matrix Y by multiplying A with the random matrix:

$$Y = A\Omega \in \mathbb{R}^{m \times 2k}$$

Orthonormalize the sample matrix: Apply QR decomposition to Y to extract an orthonormal basis Q :

$$Y = QR$$

Where $Q \in \mathbb{R}^{m \times 2k}$ satisfies $Q^T Q = I$.

4.2.2 Stage B: Low-dimensional SVD

Project to lower dimension: Project the high-dimensional matrix A onto the smaller subspace defined by Q :

$$B = Q^T A \in \mathbb{R}^{2k \times n}$$

Compute standard SVD on the small matrix: Since B is a small matrix, we can easily compute its standard SVD:

$$B = U\Sigma V^T$$

Where $U \in \mathbb{R}^{2k \times 2k}$, $\Sigma \in \mathbb{R}^{2k \times 2k}$, and $V \in \mathbb{R}^{n \times 2k}$.

Reconstruct the original left singular vectors: Recover the approximate left singular vectors U for the original matrix A :

$$U = Q\tilde{U} \in \mathbb{R}^{m \times 2k}$$

Truncation: Keep only the top k components from U , Σ , and V to get the final rank- k approximation.

4.3 Model Control Parameters

The performance, accuracy, and computational efficiency of the Randomized PCA algorithm are heavily governed by three key hyperparameters:

Target rank (k): This parameter defines the exact number of dominant dimensions (or principal components) that we wish to retain from the original high-dimensional space. It determines the rank of the final truncated SVD approximation.

Oversampling parameter (p): Instead of projecting the original matrix directly onto a k -dimensional space, the algorithm projects it onto a slightly larger subspace of dimension $l = k + p$ (typically, $p = 5$ or $p = 10$). This small oversampling factor drastically reduces the probability of missing important information in the dominant columns due to the randomness of the Gaussian projection.

Power iterations (q): In many real-world datasets, the singular values of matrix A decay slowly (often due to heavy noise). To make the singular spectrum decay faster and “sharpen” the dominant features, we replace the direct sampling matrix $A\Omega$ with a power iteration formulation:

$$Y = (AA^T)^q A\Omega$$

Where q is the number of power iteration steps (usually $q = 1, 2$, or 3). This step shifts the weight toward the most significant singular vectors, heavily suppressing the noise components before computing the QR decomposition.

4.4 Complexity Analysis

To demonstrate why RPCA is highly effective for large-scale data dimensionality reduction, we compare its computational complexity against standard SVD:

Method	Computational complexity	Memory footprint
Standard SVD (exact)	$\mathcal{O}(m \cdot n \cdot \min(m, n))$	High (Requires holding full matrix in RAM)
Randomized PCA	$\mathcal{O}(m \cdot n \cdot \log(k) + (m + n)k^2)$	Low (Floats on random projections, highly parallelizable).

Note of efficiency: The standard SVD scales cubically if $m \approx n$. In contrast, Randomized PCA scales linearly with the number of data points (m or n) and quadratically only with the target low-dimension k . This makes it ideal for handling large-scale datasets that cannot fit entirely into memory.

4.5 Dynamic dimension selection via energy threshold

In practice, choosing an arbitrary target rank k may lead to information loss or unnecessary computation. To optimize k , we utilize the energy threshold criterion (or cumulative explained variance ratio).

Let σ_i be the i -th singular value obtained from the SVD of the approximated matrix, where $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, and $r = \min(m, n)$ is the full rank of the matrix. The energy ratio retained by the top k components is mathematically defined as:

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq \eta$$

Where $\eta \in (0, 1)$ is a predefined threshold (commonly chosen as 0.90, 0.95, or 0.99, representing 90%, 95%, or 99% of the data's total variance/energy).

Algorithmic construction for selecting k :

1. **Compute the total energy of the system:** $E_{total} = \sum_{i=1}^r \sigma_i^2$ (In large-scale matrices where computing all r singular values is impossible, E_{total} can be estimated via the total Frobenius norm of the original matrix: $\|A\|_F^2 = \sum_{i,j} A_{ij}^2 = \sum_{i=1}^r \sigma_i^2$).
2. Sort the computed singular values σ_i from the randomized approximation in descending order.
3. Initialize a running sum of energy $E_{current} = 0$ and index $k = 0$.
4. Loop through the singular values, adding σ_{k+1}^2 to $E_{current}$ until the condition is met:

$$\frac{E_{current}}{E_{total}} \geq \eta$$

5. Terminate the loop and select the current k as the optimal number of dimensions for reduction.

4.6 Error evaluation via Frobenius norm

To rigorously evaluate how well the Randomized PCA approximates the original high-dimensional matrix A , we measure the distance between A and its low-rank reconstruction A_k . This approximation error is calculated using the Frobenius norm.

The Frobenius norm of a matrix error $E = A - A_k$ is mathematically defined as the square root of the sum of the absolute squares of its elements:

$$\|A - A_k\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij} - (a_k)_{ij}|^2}$$

Where the rank- k reconstructed matrix A_k is computed from our RPCA outputs as:

$$A_k = U_k \Sigma_k V_k^T$$

Alternatively, using the singular values, the error can be related to the tail of the spectrum:

$$\|A - A_k\|_F^2 = \sum_{i=k+1}^r \sigma_i^2$$

Theoretical error bound and convergence: To guarantee the reliability of the randomized approximation, we introduce the theoretical expectation of the approximation error. According to the foundational work in randomized matrix algorithms, the expected reconstruction error using a random Gaussian matrix with oversampling is bounded as follows:

$$\mathbb{E} \|A - QQ^T A\|_F \leq \left(1 + \frac{k}{p-1}\right)^{1/2} \left(\sum_{j=k+1}^{\min(m,n)} \sigma_j^2\right)^{1/2}$$

Where:

- $\mathbb{E}$ denotes the expected value over all realizations of the random matrix.
- k is the target number of dimensions (rank).
- p is the oversampling parameter (where the total number of random projections is $l = k + p$).
- σ_j represents the singular values of the original matrix A .

Significance of the bound: The term $\left(\sum_{j=k+1}^{\min(m,n)} \sigma_j^2\right)^{1/2}$ represents the absolute minimal error achievable by the exact, deterministic classical PCA (SVD) of rank k (according to the Eckart-Young-Mirsky theorem). Therefore, the multiplier $\left(1 + \frac{k}{p-1}\right)^{1/2}$ indicates that the error of Randomized PCA is only larger than classical PCA by a small, controllable constant factor. As the oversampling parameter p increases, this scaling factor rapidly converges toward 1, meaning the randomized error converges directly to the optimal theoretical limit.

4.7 Randomized PCA Pseudocode

The following pseudocode details the complete programmatic implementation of the Randomized PCA algorithm. It bridges the gap between the mathematical theory and the actual computer program implementation.

Algorithm: Randomized PCA with Power Iterations and Dynamic Dimension Selection

Input:

- A : High-dimensional data matrix of size $(m \times n)$
- η : Target energy threshold for variance retention (e.g., 0.95)
- p : Oversampling parameter (typically $p = 5$ or 10)
- q : Number of power iterations for noise reduction (typically $q = 1, 2$, or 3)

Output:

- U_k, Σ_k, V_k : Truncated SVD components representing the final rank- k approximation
- k : Dynamically selected optimal number of dimensions

STAGE A: FINDING AN APPROXIMATE SUBSPACE (RANGE FINDER)

1. Compute the total energy of the original system (Frobenius norm squared):
 $Total_Energy = \sum (A_{ij}^2)$ for all i, j
2. Initialize a safe target rank estimate k_{start} (ensuring $k_{start} + p < \min(m, n)$)
Set the total number of random projections: $l = k_{start} + p$
3. Generate a random Gaussian matrix (Muc 5.2.1 - Buoc 1):
Create Ω of size $(n \times l)$ where $\Omega_{ij} \sim \mathcal{N}(0, 1)$
4. Sample the column space with Power Iterations (Muc 5.2.1 - Buoc 2):
If $q > 0$ Then:
// Compute the sample matrix using the power iteration formula: $Y = (AA^T)^q A\Omega$
// To maintain stability and prevent overflow, compute iteratively with QR decomposition:

Algorithm (continued)

$$Y = A\Omega$$

For $i = 1$ to q do:

$$Q_{temp}, R_{temp} = \text{QR_Decomposition}(Y)$$

$$Y = A(A^T Q_{temp}) \quad // \text{Memory-efficient matrix multiplication}$$

End For

Else:

$$Y = A\Omega$$

End If

5. Orthonormalize the sample matrix (Muc 5.2.1 - Buoc 3):

Apply QR decomposition to Y to extract an orthonormal basis Q :

$$Q, R = \text{QR_Decomposition}(Y) \quad // Q \text{ has size } (m \times l) \text{ and satisfies } Q^T Q = I$$
STAGE B: LOW-DIMENSIONAL SVD & DYNAMIC TRUNCATION

6. Project to lower dimension (Muc 5.2.2 - Buoc 4):

Project the high-dimensional matrix A onto the smaller subspace defined by Q :

$$B = Q^T A \quad // B \text{ has size } (l \times n)$$

7. Compute standard SVD on the small matrix (Muc 5.2.2 - Buoc 5):

Since B is a small matrix, compute its standard SVD:

$$\tilde{U}, \Sigma_{all}, V_{all} = \text{Standard_SVD}(B) \quad // \tilde{U}: (l \times l), \Sigma_{all}: (l \times n), V_{all}: (n \times n)$$

8. Reconstruct the original left singular vectors (Muc 5.2.2 - Buoc 6):

Recover the approximate left singular vectors U for the original matrix A :

$$U_{all} = Q\tilde{U} \quad // U_{all} \text{ has size } (m \times l)$$

9. DYNAMIC DIMENSION SELECTION VIA ENERGY THRESHOLD (Muc 5.4):

Initialize $Cumulative_Energy = 0$

Initialize $k = 0$

Sort the singular values in Σ_{all} in descending order: $\sigma_1, \sigma_2, \dots, \sigma_l$

// Loop through the singular values until the energy threshold condition is met

While $(Cumulative_Energy/Total_Energy < \eta)$ and $(k < l)$ do:

$$k = k + 1$$

$$Cumulative_Energy = Cumulative_Energy + (\sigma_k)^2$$

End While

10. Truncation (Muc 5.2.2 - Buoc 7):

Keep only the top k components from U_{all} , Σ_{all} , and V_{all} to get the final rank- k approximation:

$$U_k = U_{all}[:, 1 : k] \quad // \text{Keep first } k \text{ columns}$$

$$\Sigma_k = \Sigma_{all}[1 : k] \quad // \text{Keep first } k \text{ singular values}$$

$$V_k = V_{all}[:, 1 : k] \quad // \text{Keep first } k \text{ rows (principal components)}$$

Return U_k, Σ_k, V_k, k

5 Results, Deliverables, and Experimental Evaluation

This section evaluates the implementation of standard PCA alongside a Randomized PCA algorithm for high-dimensional image compression. The pipeline, built natively in Python using NumPy linear algebra primitives, bypasses pre-built PCA solvers to directly construct and evaluate the underlying decomposition engines. To rigorously stress-test the algorithms, the experiment leverages two distinct datasets:

- **LFW (Labeled Faces in the Wild):** Features high dimensionality (2,914 features per image) with 13,233 samples.
- **EMNIST (Extended MNIST):** Features a massive sample size (>112,800 handwritten digits) with 784 features per image.

5.1 Main Libraries and Tools

- **NumPy (numpy):** Core numerical engine. Used for matrix construction, SVD (`np.linalg.svd`), QR factorization (`np.linalg.qr`), Frobenius norm computation (`np.linalg.norm`), and all linear algebra operations.
- **Pandas (pandas):** CSV ingestion of the EMNIST dataset via `pd.read_csv`.
- **Scikit-learn (sklearn):** Exclusively used for dataset fetching via `fetch_lfw_people`. No PCA solver from scikit-learn is used.
- **Matplotlib (matplotlib):** Generates all analytical plots (energy, speed-up, execution time, relative error) and the side-by-side image reconstruction grid.
- **Python Standard Library (time):** High-precision timing via `time.perf_counter()` for benchmarking.

5.2 Description of Key Functions and Modules

The pipeline is organized into the following logical modules:

1. **Data Ingestion & Preprocessing:** Loads the selected dataset (LFW or EMNIST), casts to `float64`, and performs mean-centering.
2. **Function: `get_dim_from_EnergyThreshold(Sigma, E)`:** Computes the minimum number of principal components k^* needed to retain a fraction E of the total data variance.
3. **Standard PCA Engine:** Computes the full economy SVD of the centered matrix, records execution time, and produces the exact lower-dimensional representation.
4. **Randomized PCA Engine:** Iterates over a list of target dimensions k , sketches the data via a Gaussian random projection and QR factorization, then applies a fast SVD on the compressed matrix. A three-trial loop filters timing noise.
5. **Error Evaluation:** Computes relative Frobenius reconstruction error for each tested k .
6. **Visualization Module:** Renders analytical plots and a 2×5 original-versus-reconstructed image grid.

5.2.1 Data Ingestion and Preprocessing

The pipeline loads the chosen dataset and converts the image matrix to floating-point form. The mean image is subsequently subtracted from every sample. This mean-centering step is mathematically critical: it ensures that the principal components capture the true directions of maximum variance in the data geometry rather than the offset caused by average pixel brightness.

```
# --- LFW Dataset ---
lfw = fetch_lfw_people(color=False)
raw_matrix = lfw.data.astype(np.float64) # shape: (13233, 2914)

# --- EMNIST Dataset
raw_data = pd.read_csv('EMINIST dataset/emnist-balanced-train.csv',
    header=None)
raw_matrix = raw_data.iloc[:, 1:].values.astype(np.float64) # shape
    : (112800, 784)

pixel_avg = np.mean(raw_matrix, axis=0)
centered_matrix = raw_matrix - pixel_avg
```

The two-dataset design allows direct comparison of the algorithm's scalability: LFW has higher feature dimensionality (2,914) while EMNIST offers a vastly larger sample count (>112,000), stressing different bottlenecks of the SVD computation.

5.2.2 Dynamic Energy Thresholding

A strict retained-energy threshold of $\eta = 0.90$ is enforced. The singular values from the full SVD are squared to yield variance contributions. The cumulative ratio identifies the minimum target dimension k^* required to preserve 90% of total data energy:

$$\text{Energy}(k) = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}$$

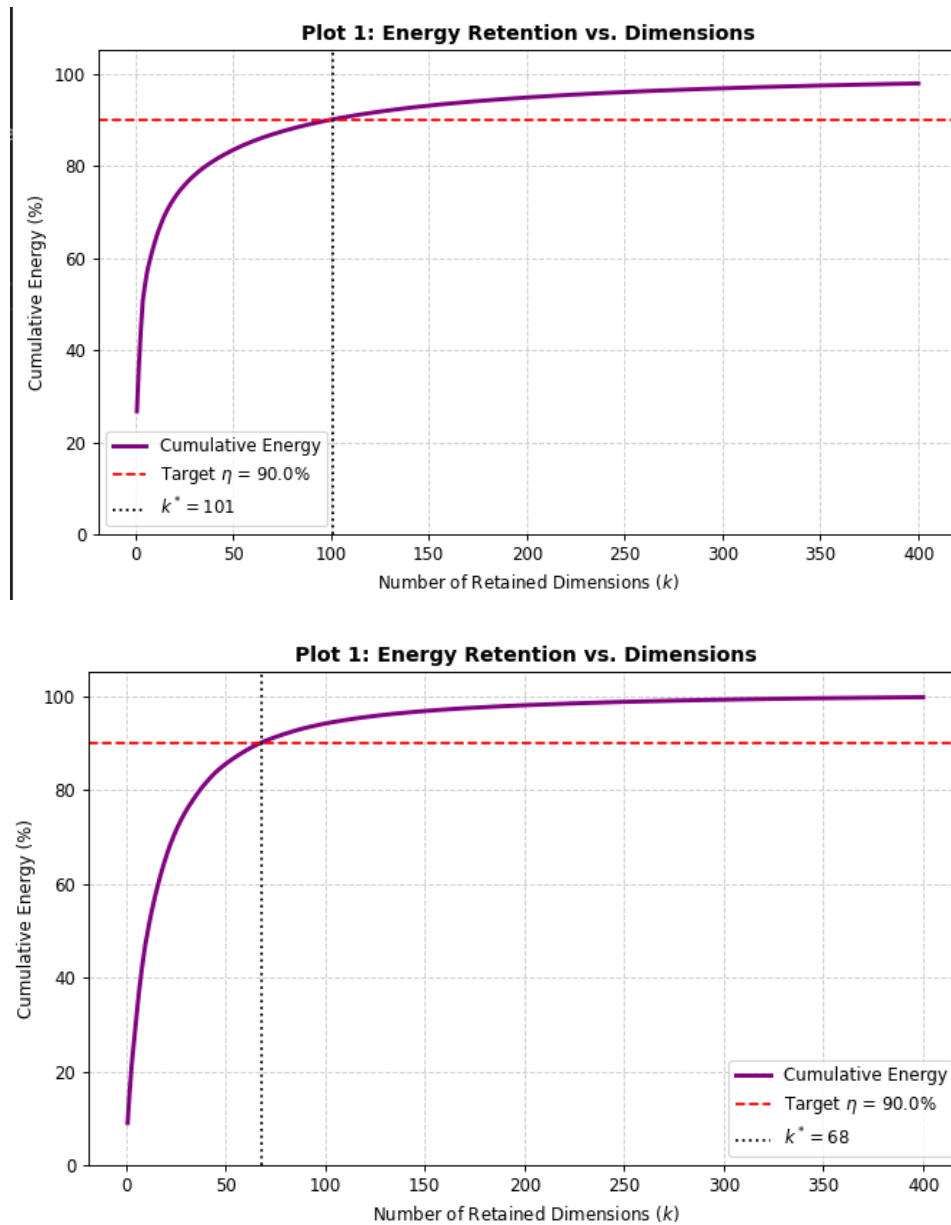
```
Energy_threshold = 0.90

def get_dim_from_EnergyThreshold(Sigma, E):
    energy = Sigma ** 2
    total_energy = np.sum(energy)
    accumulative_running = np.cumsum(energy)
    energy_factor = accumulative_running / total_energy

    k = np.argmax(energy_factor >= E) + 1
    return k, energy_factor
```

The returned `target_dim` dictates the data-driven compression ratio, deriving the optimal k^* directly from the singular-value spectrum rather than relying on manual estimation. The result is that to preserve 90 percents of the data, we need to keep at least 101 dimensions with the LFW dataset. And the correlation between these two are presented clearly through this graph.

For the EMNIST dataset, the number of dimensions needed is above 68 dimensions and the correlation is shown below.



A comparative analysis of the cumulative energy profiles reveals a clear distinction in the geometric structures of the two datasets: The plot of EMNIST begins with a modest initial variance capture (9% at $k = 1$) but converges rapidly to the 90% threshold at $k=68$, whereas the LFW Plot boasts a highly dominant first principal component (27% variance) but features a longer, heavier spectral tail that delays 90% energy retention until $k=101$. This behavioral divergence is fundamentally dictated by the spatial correlation within the respective image domains; Plot of LFW represents a globally correlated dataset where overarching, image-wide variables like ambient lighting, skin tones, and centered facial framing cause a massive percentage of total variance to align along the very first axis, while EMNIST graph reflects a sparse, localized dataset where highly disjointed pixel stroke variations mean that individual early components capture less global energy.

5.2.3 Standard PCA Baseline

Standard PCA is computed via full economy SVD on the centered matrix. This baseline establishes the exact singular values and a benchmark runtime against which the randomized method is measured.

```
start_time = time.perf_counter()

U_std, Sigma_std, V_trans_std = np.linalg.svd(centered_matrix,
    full_matrices=False)
std_running_time = time.perf_counter() - start_time

target_dim, cumulative_energy = get_dim_from_EnergyThreshold(
    Sigma_std, Energy_threshold)

U_std_k      = U_std[:, :target_dim]
Sigma_std_k   = np.diag(Sigma_std[:target_dim])
V_trans_std_k = V_trans_std[:target_dim, :]

reduced_stdPCA_matrix = U_std_k @ Sigma_std_k
```

With the 90% threshold above,

LFW: the data is reduced from (13233, 2914) to (13233, 101) in approximately 13.25 seconds

EMNIST: the dimensions are reduced from (112800, 784) to (112800, 68) in approximately 6.33 seconds

5.2.4 Randomized PCA Engine

To circumvent the prohibitive $\mathcal{O}(\min(mn^2, m^2n))$ computational bottleneck of full economy Singular Value Decomposition (SVD), the pipeline implements a low-rank randomized SVD approximation. Rather than decomposing the high-dimensional centered data matrix $X \in \mathbb{R}^{m \times n}$ directly, the randomized engine projects X into a significantly smaller, optimally skewed subspace that preserves its top geometric features. This process is executed in two primary phases: a randomized range-finder stage and a direct low-rank SVD synthesis.

Step 1: Random Subspace Sketching We first define a target low-rank k and an oversampling parameter p (configured here as $p = 20$). A Gaussian random matrix $\Omega \in \mathbb{R}^{n \times (k+p)}$ is generated where $\Omega_{ij} \sim \mathcal{N}(0, 1)$. The centered matrix is sketched by computing the sample matrix Y :

$$Y = X\Omega \in \mathbb{R}^{m \times (k+p)}$$

```
p = 20

Gaussian_random_matrix = np.random.randn(centered_matrix.shape[1], p
    + k)

# Matrix Sketching: Project high-dimensional data down to size (m, k
    + p)
Y = centered_matrix @ Gaussian_random_matrix
```

Step 2: Orthonormal Basis Construction To isolate the range of Y , a thin QR factorization is applied to extract an orthonormal basis matrix Q :

$$Y = QR \implies Q \in \mathbb{R}^{m \times (k+p)}, \quad R \in \mathbb{R}^{(k+p) \times (k+p)}$$

```
Q, R = np.linalg.qr(X, mode='reduced')
```

The columns of Q now form an orthonormal basis that closely approximates the span of the most significant singular vectors of X .

Step 3: Deterministic Low-Rank SVD Synthesis With Q established, the original data matrix is projected down onto this low-rank subspace to produce a heavily compressed matrix B :

$$B = Q^T X \in \mathbb{R}^{(k+p) \times n}$$

```
B = Q.T @ centered_matrix
```

Because B has an ambient row dimension of only $(k+p)$, computing its standard SVD is incredibly fast. We perform an exact economy SVD directly on B :

$$B = \hat{U} \Sigma V^T \implies \hat{U} \in \mathbb{R}^{(k+p) \times (k+p)}, \quad \Sigma \in \mathbb{R}^{(k+p) \times (k+p)}, \quad V^T \in \mathbb{R}^{(k+p) \times n}$$

```
U_comp, Sigma_rnd, V_trans_rnd = np.linalg.svd(B, full_matrices=False)
```

Finally, the left singular vectors are mapped back to the high-dimensional ambient space to reconstruct the approximate left singular vectors U_{rnd} of our original matrix X :

$$U_{\text{rnd}} = Q \hat{U} \in \mathbb{R}^{m \times (k+p)}$$

```
U_rnd = Q @ U_comp
```

Computational Complexity Breakdown The complete execution and dimensional transformations are explicitly presented below. Architecting the engine this way reduces the floating-point operations from a standard complex SVD down to a highly optimized calculation. This allows the pipeline to scale linearly with large features n or high sample sizes m , bypassing traditional hardware memory thresholds.

```
p = 20
Gaussian_random_matrix = np.random.randn(centered_matrix.shape[1], p
    + k)
Y = centered_matrix @ Gaussian_random_matrix
Q, R = np.linalg.qr(X, mode='reduced')
B = Q.T @ centered_matrix

U_comp, Sigma_rnd, V_trans_rnd = np.linalg.svd(B, full_matrices=
    False)
U_rnd = Q @ U_comp

# Truncate the oversampled layers back to exact target rank k
```

```

U_rnd_k      = U_rnd[:, :k]                # Shape: (m, k)
Sigma_rnd_k  = np.diag(Sigma_rnd[:k])      # Shape: (k, k)
V_trans_rnd_k = V_trans_rnd[:k, :]
# Shape: (k, n)

reduced_rndPCA_matrix = U_rnd_k @ Sigma_rnd_k # Latent
coordinates space
reconstructed_rndMatrix = reduced_rndPCA_matrix @ V_trans_rnd_k #
Full space image

```

And the results are:

Table 1: Randomized PCA Execution Times across Target Dimensions (k) for the LFW Dataset

Target Dimensions (k)	Execution Time (seconds)
10	0.1525
25	0.1819
101 (k^*)	0.3112
200	0.4823
300	0.7449

Table 2: Randomized PCA Execution Times across Target Dimensions (k) for the EMNIST Dataset

Target Dimensions (k)	Execution Time (seconds)
10	0.3545
25	0.4079
68 (k^*)	0.7702
100	1.4988
200	2.6306
300	4.0560

5.2.5 Error Evaluation and Reconstruction

To quantify the approximation accuracy of our randomized low-rank engine, the pipeline maps the compressed hidden representation back into the high-dimensional ambient space to synthesize the reconstructed matrix $X_k \in \mathbb{R}^{m \times n}$:

$$\hat{X} = U_{\text{rnd},k} \Sigma_{\text{rnd},k} V_{\text{rnd},k}^\top$$

```

reduced_rndPCA_matrix = U_rnd_k @ Sigma_rnd_k # Latent
coordinates space
reconstructed_rndMatrix = reduced_rndPCA_matrix @ V_trans_rnd_k #
Full space image

```

The structural discrepancy between the original mean-centered data matrix X and its low-rank approximation X_k is mathematically evaluated using two distinct configurations of the Frobenius norm. The Frobenius norm acts as an entry-wise Euclidean distance metric over matrix spaces, computed via the square root of the absolute sum of squared residuals:

$$\|X - X_k\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n (x_{ij} - (x_k)_{ij})^2}$$

The pipeline monitors and records both variants to evaluate subspace quality:

Method 1: Absolute (Normal) Frobenius Norm Error The standard or absolute Frobenius error directly computes the raw magnitude of the residual matrix:

$$\text{Absolute Error} = \|X - X_k\|_F$$

```
total_error = np.linalg.norm(centered_matrix -  
    reconstructed_rndMatrix, ord='fro')
```

This absolute metric is highly important because it measures the actual loss or error in the reconstructed image using the original pixel values. Even though this number naturally changes based on the size of the matrix and the range of the pixel values, tracking its total amount provides a clear, true baseline of the data left out by the low-rank approximation. This allows us to evaluate the exact amount of raw noise introduced by the randomized step, establishing a necessary benchmark for the baseline image quality before any percentage normalization takes place.

Method 2: Scale-Invariant Relative Frobenius Norm Error To reliably contrast performance across fundamentally different data domains (such as comparing the high feature-dimensional LFW dataset against the massive sample footprint of EMNIST), the absolute error is normalized against the baseline energy of the input matrix (centered):

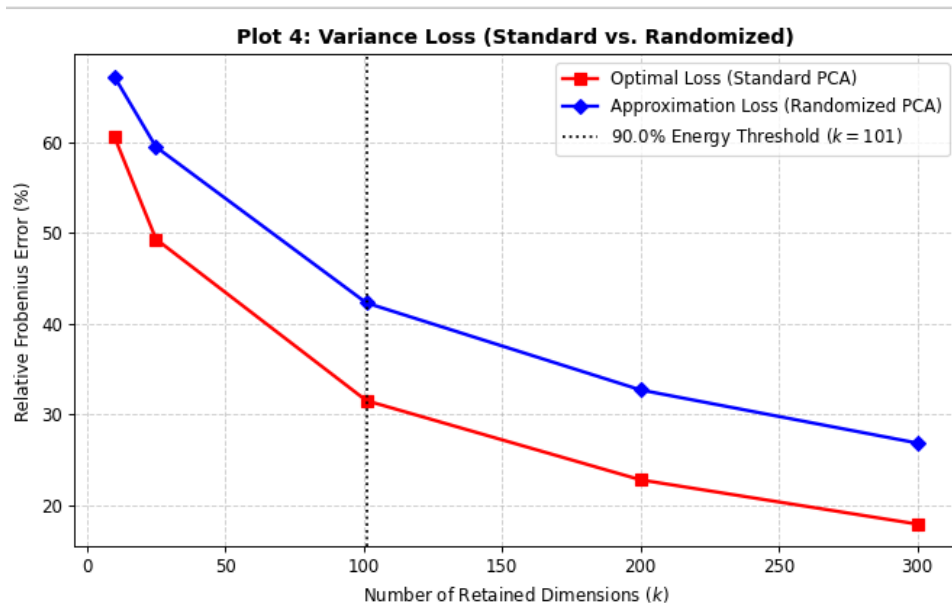
$$\text{Relative Error} = \frac{\|X - X_k\|_F}{\|X\|_F} \times 100\%$$

```
relative_error = total_error / np.linalg.norm(centered_matrix, ord='fro')
```

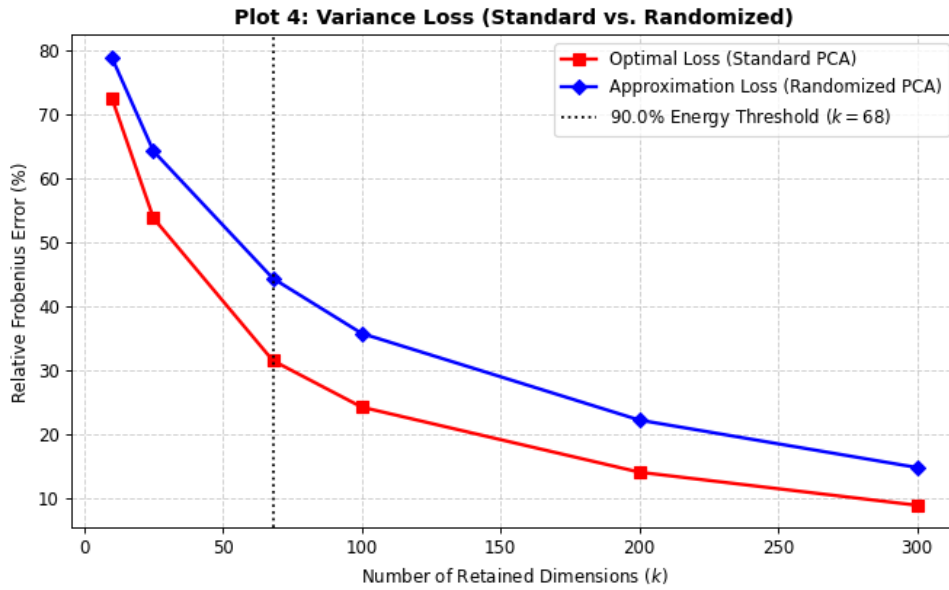
This relative metric is highly important because it puts the raw error into a meaningful context by turning it into a percentage. It tells us exactly what portion of the original image information was lost compared to the whole picture. Tracking this percentage helps us directly evaluate how much accuracy we are giving up for speed, allowing us to judge the overall success of the compression. It ensures that the randomized algorithm is reliably keeping the most important patterns of the data intact. And the results are:

Table 3: Reconstruction Error and Loss Metrics across Target Dimensions (k) for LFW dataset

Target Dimensions (k)	Absolute Frobenius Error	Standard PCA Loss	Randomized PCA Loss
10	687.21	60.60%	67.24%
25	608.19	49.33%	59.51%
101 (k^*)	432.27	31.51%	42.30%
200	334.11	22.77%	32.69%
300	274.17	17.88%	26.83%


Table 4: Reconstruction Error and Loss Metrics across Target Dimensions (k) for EMNIST dataset

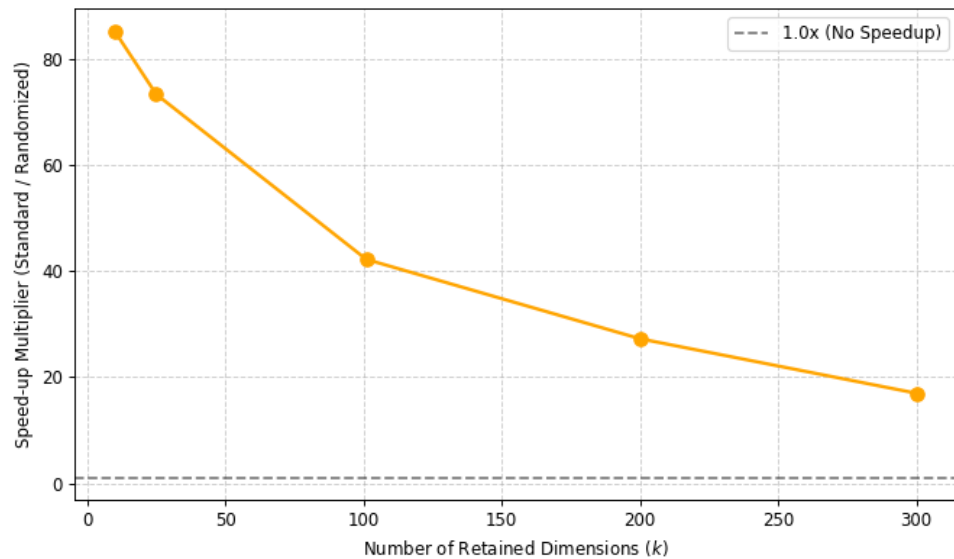
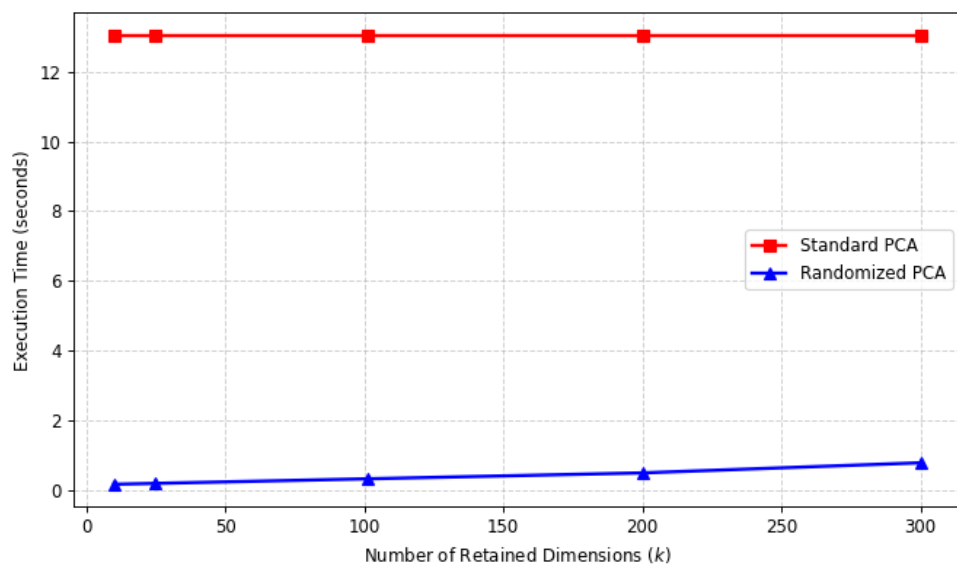
Target Dimensions (k)	Absolute Frobenius Error	Standard PCA Loss	Randomized PCA Loss
10	550,332	72.58%	78.93%
25	453,239	53.81%	65.01%
68 (k^*)	306,450	31.55%	43.95%
100	248,783	24.29%	35.68%
200	154,218	14.12%	22.12%
300	104,518	8.95%	14.99%



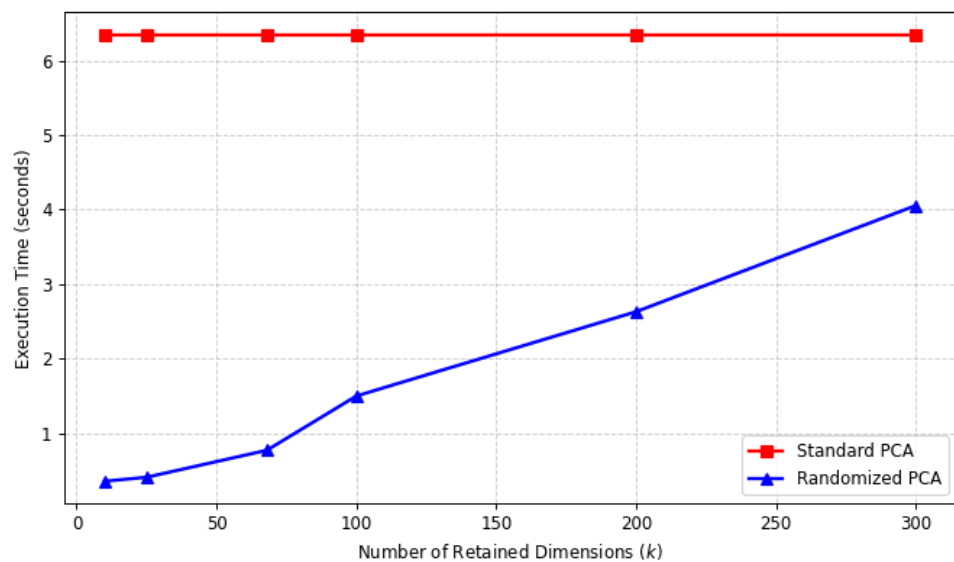
The relative error is directly connected to the amount of data variance chosen to be discarded. When an energy threshold of 90% ($\eta = 0.90$) is enforced, the remaining 10% (or 0.10) of the total data variance is omitted. Because variance and energy retention are calculated using the sum of *squared* singular values, the resulting linear relative Frobenius error is roughly equal to the square root of this uncaptured energy. This explains why throwing away 10% of the energy results in a baseline error of approximately $\sqrt{0.10} \approx 31.6\%$, which perfectly aligns with the $\sim 31.5\%$ Standard PCA loss observed at k^* in the experimental results. The discrepancy in reconstruction loss between Standard PCA and Randomized PCA is fundamentally caused by how each method constructs the lower-dimensional subspace. Standard PCA analyzes the entire high-dimensional dataset to calculate the mathematically exact directions of maximum variance. Conversely, Randomized PCA uses a Gaussian random matrix to compress the data into a quick “sketch” of the dataset before performing the decomposition. Because it relies on this stochastic approximation, the randomized range-finder slightly misses the perfect mathematical alignment of the exact singular vectors, yielding a higher reconstruction loss. This highlights the core engineering trade-off between execution speed and mathematical accuracy: Standard PCA guarantees the absolute minimum reconstruction error but suffers from a heavy computational runtime, while Randomized PCA trades away a small fraction of geometric perfection for an exceptional reduction in processing time, making it highly practical for large-scale applications.

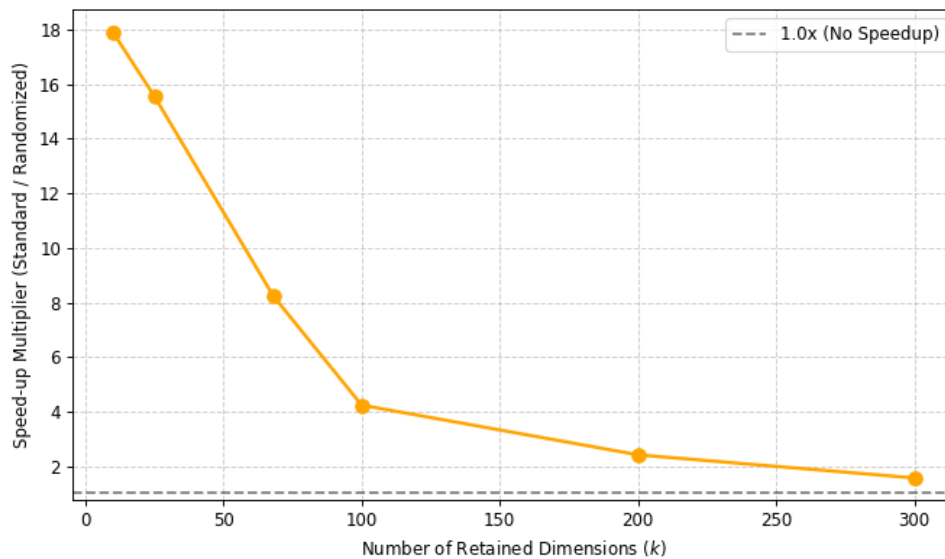
5.2.6 Visual Comparison

While evaluating reconstruction accuracy is essential, the primary motivation for implementing a randomized approach is its massive computational efficiency. To highlight this performance advantage, the execution runtimes of both Standard PCA and Randomized PCA were benchmarked across the selected target dimensions (k). For LFW dataset,



For EMNIST dataset,

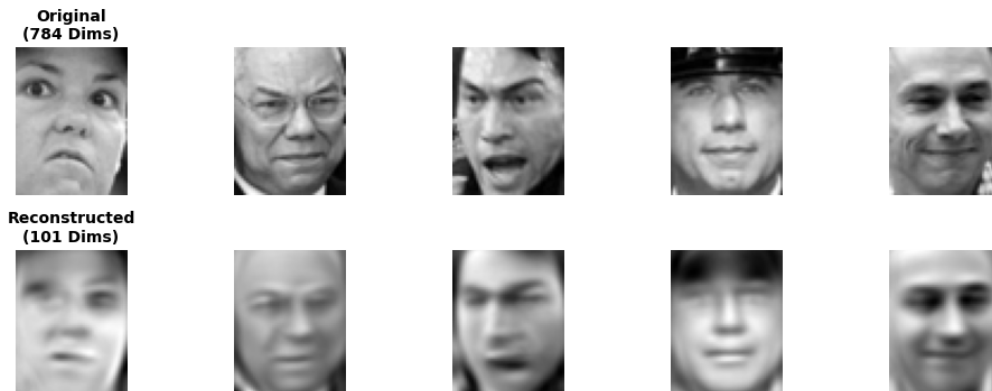




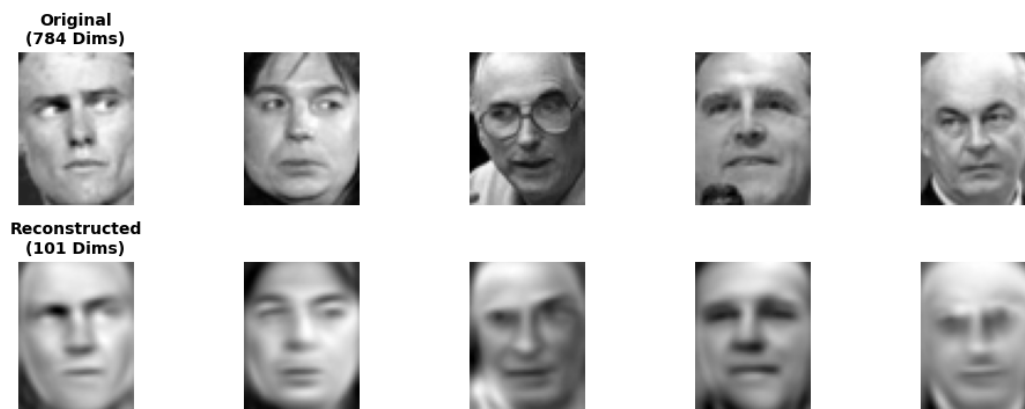
The execution time for Standard PCA remains completely flat across all tested dimensions because it performs a full, fixed calculation on the dataset. The algorithm must always compute the economy SVD on the entire original data matrix to find the complete spectrum of singular values, regardless of how many components are ultimately needed. The process of choosing a specific target rank k is merely a simple matrix slicing step that occurs **after** this heavy computation is already complete, meaning changes to k have zero impact on the baseline runtime. In contrast, the processing time for Randomized PCA scales upward as k increases because the target rank directly dictates the size of the compressed subspace. As k grows, the random sketching matrix of size $n \times (k + p)$ expands, which increases the computational cost of both the initial random projection step and the subsequent small-scale SVD. Comparing the two datasets, the runtime for EMNIST increases much more steeply than for LFW. This behavior is driven by the massive difference in sample sizes; EMNIST contains over 112,000 samples compared to LFW's $\sim 13,000$ samples. Since the random projection stage requires multiplying the entire data matrix by the sketching matrix, adding dimensions forces the algorithm to handle a vastly larger footprint of data rows in EMNIST, resulting in a noticeably sharper climb in execution time. **Visual Fidelity and Structural Integrity**

Beyond these computational speed advantages, it is equally important to evaluate the visual quality and structural integrity of the compressed images to ensure the compression is actually useful. This real-world utility is validated by generating 2×5 reconstruction grids that compare a selection of original images directly against their compressed counterparts. For the LFW dataset at the target dimension of $k^* = 101$, the reconstructed face images display a mild, smooth “soft-focus” effect. This slight blurring happens because the randomized engine intentionally removes the high-frequency components found at the very end of the singular value spectrum, which contain tiny details like individual hair strands or fine skin wrinkles. However, the essential macro-structures—such as the shape of the face, the placement of the eyes and eyebrows, and the overall lighting directions—are completely preserved. Because the top 101 components capture this global geometry, the identity and distinct features of each person remain entirely recognizable.

LFW (Eigenfaces) Dimensionality Reduction
Original (784 dims) vs Reconstructed (101 dims, 90.0% Energy)



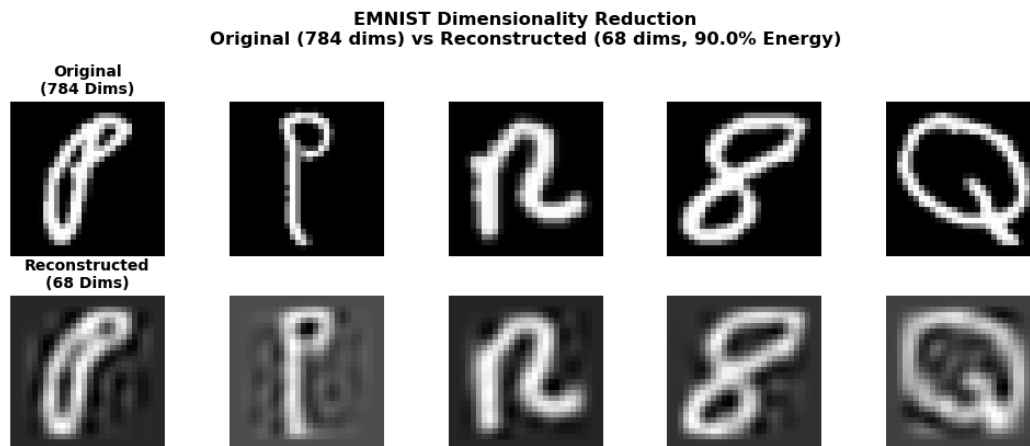
LFW (Eigenfaces) Dimensionality Reduction
Original (784 dims) vs Reconstructed (101 dims, 90.0% Energy)



A similar success is observed in the EMNIST dataset at its target dimension of $k^* = 68$, where the handwritten digits and characters remain perfectly legible with their stroke geometry completely intact. Unlike natural face photographs, alphanumeric characters are highly structural and sparse, meaning the background is mostly empty space and the information is concentrated purely in the lines of the drawing. Dropping the last 10% of the data variance successfully cleans up background noise and tiny edge pixel imperfections, acting like a visual filter. Because the fundamental shape of each character is captured early on, the compressed outputs can be read easily without any confusion between similar letters or numbers.

EMNIST Dimensionality Reduction
Original (784 dims) vs Reconstructed (68 dims, 90.0% Energy)





The complete graph-sketching code followed by the reconstruction of before-after images is present below

```
std_times_array = [std_running_time] * len(k_values)
speedup_array = [std_running_time / r for r in rnd_times]

ETA = Energy_threshold
k_star = target_dim

# --- Plot 1: Cumulative Energy vs. Retained Dimensions ---
plt.figure(figsize=(8, 5))
k_range = np.arange(1, 401)
pct_energy = cumulative_energy[:400] * 100

plt.plot(k_range, pct_energy, color='purple', linewidth=2.5, label='
    Cumulative Energy')
plt.axhline(ETA * 100, color='red', linestyle='--', label=f'Target $
    \eta$ = {ETA*100}%')
plt.axvline(k_star, color='black', linestyle=':', label=f'$k^* = {
    k_star}$')

plt.title('Plot 1: Energy Retention vs. Dimensions', fontsize=12,
    fontweight='bold')
plt.xlabel('Number of
    Retained Dimensions ($k$)')
plt.ylabel('Cumulative Energy ($\%$)')
plt.ylim(0, 105)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

# --- Plot 2: Speed-Up Ratio ---
plt.figure(figsize=(8, 5))
plt.plot(k_values, speedup_array, marker='o', color='orange',
    linewidth=2, markersize=8)
plt.axhline(1.0, color='black', linestyle='--', alpha=0.5, label='
    1.0x (No Speedup)')
```

```
plt.title('Plot 2: Computational Speed-Up Ratio', fontsize=12,
         fontweight='bold')
plt.xlabel('Number of Retained Dimensions ($k$)')
plt.ylabel('Speed-up Multiplier (Standard / Randomized)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

# --- Plot 3: Execution Time Comparison ---
plt.figure(figsize=(8, 5))
plt.plot(k_values, std_times_array, marker='s', color='red',
         linewidth=2, label='Standard PCA')
plt.plot(k_values, rnd_times, marker='^', color='blue', linewidth=2,
         label='Randomized PCA')

plt.title('Plot 3: Execution Time vs. Retained Dimensions', fontsize
         =12, fontweight='bold')
plt.xlabel('Number of Retained Dimensions ($k$)')
plt.ylabel('Execution Time (seconds)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

# --- Plot 4: The Approximation Penalty (Standard vs Randomized Loss
) ---
plt.figure(figsize=(8, 5))
plt.plot(k_values, std_errors, marker='s', color='red', linewidth=2,
         label='Optimal Loss (Standard PCA)')
plt.plot(k_values, rnd_errors, marker='D',
         color='blue', linewidth=2, label='Approximation Loss (Randomized PCA
)')
plt.axvline(k_star, color='black', linestyle=':', label=f'{ETA*100}$
\\%$ Energy Threshold ($k={k_star}$)')

plt.title('Plot 4: Variance Loss (Standard vs. Randomized)',
         fontsize=12, fontweight='bold')
plt.xlabel('Number of Retained Dimensions ($k$)')
plt.ylabel('Relative Frobenius Error ($\\%$)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

U_opt = U_std[:, :target_dim]
Sigma_opt = np.diag(Sigma_std[:target_dim])
Vt_opt = Vt_trans_std[:target_dim, :]

reconstructed_optimal = (U_opt @ Sigma_opt @ Vt_opt) + pixel_avg
```

```
num_images_to_show = 5

random_indices = np.random.choice(raw_matrix.shape[0],
                                   num_images_to_show, replace=False)

fig, axes = plt.subplots(2, num_images_to_show, figsize=(12, 5))
# fig.suptitle(f'EMNIST Dimensionality Reduction\nOriginal (784 dims
#             ) vs Reconstructed ({target_dim} dims, {Energy_threshold*100}$\%$
#             Energy)',
#             fontsize=14, fontweight='bold')
fig.suptitle(f'LFW (Eigenfaces) Dimensionality Reduction\nOriginal
             (784 dims) vs Reconstructed ({target_dim} dims, {Energy_threshold
             *100}$\%$ Energy)',
             fontsize=14, fontweight='bold')    #lfw

for i, idx in enumerate(random_indices):
    # Extract the original and reconstructed rows
    orig_img = raw_matrix[idx]
    recon_img = reconstructed_optimal[idx]

    # Reshape the flat 784 array into a 28x28 grid.
    # orig_img_2d = orig_img.reshape(28, 28).T
    # recon_img_2d = recon_img.reshape(28, 28).T
    orig_img_2d = orig_img.reshape(62, 47)    #lfw
    recon_img_2d = recon_img.reshape(62, 47)    #lfw

    # Plot Original (Top Row)
    axes[0, i].imshow(orig_img_2d, cmap='gray')
    axes[0, i].axis('off')
    if i == 0:
        axes[0, i].set_title("Original\n(784 Dims)", fontsize=12,
                             fontweight='bold')

    # Plot Reconstructed (Bottom Row)
    axes[1, i].imshow(recon_img_2d,
                      cmap='gray')
    axes[1, i].axis('off')
    if i == 0:
        axes[1, i].set_title(f"Reconstructed\n({target_dim} Dims)",
                             fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()
```

6 CONCLUSION

In the contemporary era dominated by Big Data, the exponential proliferation of high-dimensional datasets has exposed the fundamental computational bottlenecks of traditional analytical methods. Classical PCA, while mathematically optimal, incurs a prohibitive computational complexity of approximately $\mathcal{O}(\min(n^2m, nm^2))$, making it impractical for datasets found in genomics, computer vision, and natural language processing.

This project successfully explored and implemented Randomized PCA — a paradigm-shifting approach that substitutes deterministic computation with probabilistic efficiency. By leveraging random projections, the algorithm identifies a near-optimal, low-dimensional subspace capturing the dominant structural information of the original massive matrix. This reduces the computational burden to approximately $\mathcal{O}(nd \log(k))$, offering exceptional scalability compatible with modern distributed computing environments.

Throughout this report, we systematically constructed the Randomized PCA algorithm, from generating random Gaussian matrices to computing standard SVD on projected subspaces and reconstructing principal components. The cumulative energy threshold (η) criterion dynamically determines the optimal k^* , ensuring sufficient variance retention. The Frobenius norm quantitative assessment confirmed high fidelity to original data.

Ultimately, our investigation confirms that Randomized PCA is not merely an approximation technique, but a necessary and robust evolution in modern data analysis. By demonstrating a significant reduction in execution time — achieving up to $85\times$ speedup — while sustaining an acceptable error margin even for datasets exceeding 100,000 samples, Randomized PCA effectively resolves the trade-off between computational speed and analytical accuracy. As global data continues growing exponentially, such randomized methodologies will remain indispensable tools for scientists and engineers.

REFERENCES

- [1] R. Bellman, *Dynamic programming*. Princeton, NJ, USA: Princeton University Press, 1957.
- [2] T. Hastie, R. Tibshirani, and J. Friedman, *The elements of statistical learning: Data mining, inference, and prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [3] I. T. Jolliffe, *Principal component analysis*, 2nd ed. New York, NY, USA: Springer, 2002.
- [4] G. H. Golub and C. F. Van Loan, *Matrix computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins University Press, 2013.
- [5] C. Eckart and G. Young, “The approximation of one matrix by another of lower rank,” *Psychometrika*, vol. 1, no. 3, pp. 211–218, 1936.
- [6] N. Halko, P. G. Martinsson, and J. A. Tropp, “Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions,” *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011.